Assignment No. 11

- **TITLE –** Solve problems like Tower of Hanoi and Factorial using both recursion and explicit stack simulation to understand how function calls are managed internally. Compare recursion depth, space usage, and manually simulate the call stack to visualize execution flow.

- **THEORY –**

1. **Recursion**
   Recursion is a programming technique where a function calls itself to solve smaller instances of the same problem. It relies on the system's call stack to manage function calls and local variables.
2. **Explicit Stack Simulation**
   This approach manually manages the execution flow using a stack data structure, mimicking how the system handles recursive function calls internally.

- **Tower of Hanoi -** Move n disks from source peg to destination peg using an auxiliary peg, following:
  1. Only one disk can be moved at a time
  2. Only the top disk can be moved
  3. No larger disk can be placed on top of a smaller disk

1. Recursive Solution

```
Algorithm HanoiRecursive(n, source, dest, aux):
1. IF n = 1:
      PRINT "Move disk from source to dest"
      RETURN
2. HanoiRecursive(n-1, source, aux, dest)
3. PRINT "Move disk n from source to dest"
4. HanoiRecursive(n-1, aux, dest, source)
```

Time Complexity: $O(2^n)$
Space Complexity: O(n) – recursion depth

2. Explicit Stack Simulation

```
Algorithm HanoiIterative(n, source, dest, aux):
1. CREATE stack
2. PUSH (n, source, dest, aux) to stack
3. WHILE stack is not empty:
      POP (num, src, dst, ax) from stack
      IF num = 1:
         PRINT "Move disk from src to dst"
      ELSE:
         PUSH (num-1, ax, dst, src)  // Second recursive call
         PUSH (1, src, dst, ax)      // Move largest disk
         PUSH (num-1, src, ax, dst)  // First recursive call
4. END
```

Time Complexity: $O(2^n)$
Space Complexity: O(n) - explicit stack size

- **Factorial Calculation**

1. Recursive Solution

Algorithm FactorialRecursive(n):
1. IF n = 0 OR n = 1:
   RETURN 1
2. RETURN n * FactorialRecursive(n-1)
Time Complexity: O(n)
Space Complexity: O(n) - recursion depth

2. Explicit Stack Simulation

Algorithm FactorialIterative(n):
1. CREATE stack
2. result ← 1
3. PUSH n to stack
4. WHILE stack is not empty:
   POP current
   result ← result * current
   IF current > 1:
      PUSH current-1
5. RETURN result

➢ CODE –

```
#include <iostream>
using namespace std;
class Stack {
private:
    int* arr;
    int top;
    int capacity;
public:
    Stack(int size) {
        capacity = size;
        arr = new int[capacity];
        top = -1;
    }
    ~Stack() {
        delete[] arr;
    }
    void push(int value) {
        if (top == capacity - 1) {
            cout << "Stack overflow" << endl;
            return;
        }
        arr[++top] = value;
    }
    int pop() {
        if (isEmpty()) {
            cout << "Stack underflow" << endl;
```

```
            return -1;
        }
        return arr[top--];
    }
    int peek() {
        if (isEmpty()) {
            return -1;
        }
        return arr[top];
    }
    bool isEmpty() {
        return top == -1;
    }
    int size() {
        return top + 1;
    }
};
void hanoiRecursive(int n, char from, char to, char aux) {
    if (n == 1) {
        cout << "Move disk 1 from " << from << " to " << to << endl;
        return;
    }
    hanoiRecursive(n - 1, from, aux, to);
    cout << "Move disk " << n << " from " << from << " to " << to << endl;
    hanoiRecursive(n - 1, aux, to, from);
}
void hanoiStack(int n, char from, char to, char aux) {
    Stack st(1000);
    st.push(n);
    st.push(from);
    st.push(to);
    st.push(aux);
    st.push(0);
    while (!st.isEmpty()) {
        int stage = st.pop();
        char a = st.pop();
        char t = st.pop();
        char f = st.pop();
        int num = st.pop();
        if (num == 1) {
            cout << "Move disk 1 from " << f << " to " << t << endl;
            continue;
        }
        if (stage == 0) {
            st.push(num);
            st.push(f);
            st.push(t);
            st.push(a);
            st.push(1);
            st.push(num - 1);
            st.push(f);
            st.push(a);
            st.push(t);
            st.push(0);
        } else if (stage == 1) {
            cout << "Move disk " << num << " from " << f << " to " << t << endl;
            st.push(num - 1);
            st.push(a);
            st.push(t);
            st.push(f);
```

```
            st.push(0);
        }
    }
}
int factorialRecursive(int n) {
    if (n == 0 || n == 1) return 1;
    return n * factorialRecursive(n - 1);
}
int factorialStack(int n) {
    Stack st(100);
    int result = 1;
    while (n > 1) {
        st.push(n);
        n--;
    }
    while (!st.isEmpty()) {
        result *= st.pop();
    }
    return result;
}
int main() {
    int choice1, choice2;
    cout << "Choose method:\n1. Recursion\n2. Stack\n";
    cin >> choice1;
    cout << "Choose operation:\n1. Tower of Hanoi\n2. Factorial\n";
    cin >> choice2;
    if (choice2 == 1) {
        int n;
        cout << "Enter number of disks: ";
        cin >> n;
        if (choice1 == 1) {
            hanoiRecursive(n, 'A', 'C', 'B');
        } else {
            hanoiStack(n, 'A', 'C', 'B');
        }
    } else {
        int n;
        cout << "Enter number: ";
        cin >> n;
        if (choice1 == 1) {
            cout << "Factorial: " << factorialRecursive(n) << endl;
        } else {
            cout << "Factorial: " << factorialStack(n) << endl;
        }
    }
    return 0;
}
```

- **OUTPUT –**

Choose method:
1. Recursion
2. Stack
1
Choose operation:
1. Tower of Hanoi
2. Factorial
1
Enter number of disks: 4

```
Move disk 1 from A to B
Move disk 2 from A to C
Move disk 1 from B to C
Move disk 3 from A to B
Move disk 1 from C to A
Move disk 2 from C to B
Move disk 1 from A to B
Move disk 4 from A to C
Move disk 1 from B to C
Move disk 2 from B to A
Move disk 1 from C to A
Move disk 3 from B to C
Move disk 1 from A to B
Move disk 2 from A to C
Move disk 1 from B to C

Choose method:
1. Recursion
2. Stack
1
Choose operation:
1. Tower of Hanoi
2. Factorial
2
Enter number: 4
Factorial: 24

Choose method:
1. Recursion
2. Stack
2
Choose operation:
1. Tower of Hanoi
2. Factorial
1
Enter number of disks: 4
Move disk 1 from A to B
Move disk 2 from A to C
Move disk 1 from B to C
Move disk 3 from A to B
Move disk 1 from C to A
Move disk 2 from C to B
Move disk 1 from A to B
Move disk 4 from A to C
Move disk 1 from B to C
Move disk 2 from B to A
Move disk 1 from C to A
Move disk 3 from B to C
Move disk 1 from A to B
Move disk 2 from A to C
Move disk 1 from B to C
```

Choose method:
1. Recursion
2. Stack
2
Choose operation:
1. Tower of Hanoi
2. Factorial
2
Enter number: 4
Factorial: 24

- **CONCLUSION –**

This practical lab demonstrates how stack and recursion can be used to solve the problem of tower of Hanoi and to compute factorial of a number.